ACADEMIC LABORATORY PORTFOLIO & TECHNICAL REPORT

ARTIFICIAL INTELLIGENCE

Course Code: **IC-709D** | Implementation Runtime: **Python 3.8+**

Submitted By:

JEET VERMA

Roll Number: IC-2K23-46

Department of Computer Science & Engineering

International Institute of Professional Studies (IIPS)

Devi Ahilya Vishwavidyalaya (DAVV), Indore, Madhya Pradesh, India

Academic Session: Academic Year 2026–2027

Repository Identity: `IC-2K23-46-JEET-VERMA`

A comprehensive compilation of practical implementations, theoretical formulations, complexity analysis, algorithmic flowcharts, and empirical observations across all seven prescribed laboratory units.

Table of Contents

Section	Title & Subsections
1	Cover Page & Academic Affiliation
2	Table of Contents
3	Introduction to Artificial Intelligence (Symbolic vs Sub-symbolic AI)
4	Objectives of the Laboratory (Curricular Competencies)
5	Programming Environment, Tools and Python Runtimes
6	Overview of the Seven Laboratory Units (System Architecture)
7	Detailed Unit-wise Experiments (Labs 1 through 7 In-Depth)
7.1	Unit 1: AI Problem Solving & State Spaces (Water Jug, Vacuum, M&C;, 8-Puzzle)
7.2	Unit 2: Search Algorithms & Navigation (8-Puzzle BFS, Graph Traversal, Maze, Route)
7.3	Unit 3: AI Frameworks & Data Preparation (Imputation, Encoding, Scaling, ANOVA, MI)
7.4	Unit 4: Knowledge Representation & Logical Reasoning (Predicates, SemNets, Frames, Rules)
7.5	Unit 5: Decision Tree Classifier (Campus Placement CART, Leakage Guard, Depth Tuning)
7.6	Unit 6: Deep Learning using PyTorch (Tensors, Autograd DAGs, Regression, Moons MLP)
7.7	Unit 7: Explainable AI, Adversarial AI & Advanced Applications (Grad-CAM, FGSM, PCA/SVD)
8	AI Concepts and Theoretical Foundations
9	Problem Formulation and State-space Representation
10	Search Strategies and Algorithmic Approaches (Uninformed vs Informed Trade-offs)
11	Knowledge Representation and Reasoning Systems
12	Machine Learning and Deep Learning Methodologies
13	Important Diagrams and Algorithm Flowcharts
14	Complexity Analysis (Space-Time Bounds, Branching Factors, Optimality)
15	Experimental Observations and Quantitative Results
16	Challenges Encountered and Engineering Solutions
17	Learning Outcomes & Skill Mastery
18	Conclusion & Methodological Synthesis
19	References and Authoritative Literature

3. Introduction to Artificial Intelligence

Artificial Intelligence (AI) is the rigorous scientific discipline devoted to constructing computational agents that perceive their environments, formulate rational goals, represent domain knowledge symbolically, deduce valid inferences, and adaptively learn from empirical observation. Historically, AI has evolved across two complementary paradigms:

1. Symbolic Artificial Intelligence (Classical AI): Founded upon the Physical Symbol System Hypothesis (Newell & Simon, 1976), symbolic AI posits that intelligent action is achieved through explicit manipulation of symbolic tokens governed by formal logic, semantic graphs, object-centered frames, and declarative production rules. Classical AI guarantees mathematical soundness, transparency, and human auditability, but historically faced challenges when dealing with high-dimensional sensory noise.

2. Sub-symbolic and Differentiable AI (Modern Machine Learning & Deep Learning): Rather than programming hand-crafted rules, modern sub-symbolic systems approximate complex continuous functions by optimizing parameterized weight tensors through gradient-based backpropagation over dynamic computational graphs. While achieving state-of-the-art predictive performance in vision and speech, deep neural architectures operate largely as 'black-box' estimators, introducing critical requirements for *Explainable AI (XAI)* and *Adversarial Robustness Auditing*.

The coursework in **IC-709D: Artificial Intelligence** systematically traverses both paradigms. The laboratory begins with discrete state-space formulations and uninformed search algorithms, advances through formal knowledge engineering, and culminates in leak-free tabular machine learning, deep neural network training in PyTorch, convolutional feature explainability (Grad-CAM), adversarial vulnerability analysis (FGSM), and matrix-factorized dimensionality reduction (PCA & SVD).

4. Objectives of the Laboratory

The primary curricular objective of the IC-709D laboratory is to cultivate comprehensive theoretical and empirical competency in the design, implementation, analysis, and validation of artificial intelligence algorithms. Specifically, students must demonstrate:

- **Rigorous Problem Formulation:** Translating real-world physical and combinatorial puzzles into formal state spaces $\langle S, s_0, \mathcal{O}, G \rangle$ with explicit transition constraints.
- **Systematic Search Exploration:** Implementing Breadth-First Search (BFS) and Depth-First Search (DFS) using standard data structures (`collections.deque`, recursion), verifying completeness and shortest-path optimality.
- **Robust Data Engineering:** Designing leak-free tabular preprocessing workflows that prevent catastrophic target leakage and preserve evaluation integrity across stratified train-test partitions.
- **Declarative Knowledge Modeling:** Engineering symbolic knowledge bases, semantic inheritance graphs, Minsky frames, and forward-chaining rule engines using first-principles Python.
- **Interpretable Machine Learning:** Training white-box CART decision tree classifiers, inspecting Gini impurity reduction, and regularizing model variance through depth pre-pruning.
- **Differentiable Neural Programming:** Mastering PyTorch automatic differentiation (`torch.autograd`), constructing parametric neural layers (`torch.nn.Module`), and executing canonical 5-step training loops.
- **Modern AI Safety & Interpretability:** Implementing Grad-CAM hooks to visualize CNN spatial attention on CIFAR-10, auditing adversarial fragility under FGSM attacks, and proving mathematical PCA $\equiv$ SVD equivalence.

5. Programming Environment, Tools and Python Runtimes

All algorithms, models, and analytical pipelines in this laboratory were implemented in native **Python 3.8+** on Linux x86_64. The implementations deliberately leverage open-source, reproducible libraries without proprietary dependencies:

Tool / Framework	Version Role	Curricular Function in Laboratory
Python 3	>= 3.10	Core runtime language for all algorithms, search engines, and symbolic structures.
PyTorch (`torch`)	2.15.0+cpu	Tensors, dynamic autograd computation graphs, neural layers, hooks, and loss optimization.
Torchvision	0.29.2+cpu	Computer vision dataset loading (CIFAR-10) and tensor image transformation pipelines.
NumPy	2.5.3	High-performance n-dimensional array manipulation, linear algebra, and SVD decomposition.
Pandas	3.0.6	Tabular schema profiling, missing-value imputation, one-hot encoding, and CSV serializations.
Scikit-Learn	1.9.1	ColumnTransformer preprocessing, ANOVA F-test, Mutual Info, CART Decision Trees, Iris PCA.
Matplotlib / Seaborn	3.11.2 / 0.13.2	Statistical EDA visualizations, confusion matrix heatmaps, training loss curves, and Grad-CAM overlays.

6. Overview of the Seven Laboratory Units

The laboratory repository is structured into seven modular units, progressing from foundational search to cutting-edge explainability:

- **Unit 1 — AI Problem Solving:** Formulating state-space graphs, operators, and visited-state BFS exploration across Water Jug, Vacuum Cleaner, Missionaries & Cannibals, and 8-Puzzle tile movements.
- **Unit 2 — Search Algorithms and Navigation:** Systematic graph search (BFS and DFS), recursive obstacle maze pathfinding, grid-based robot navigation, and shortest-path road route planning.
- **Unit 3 — AI Programming Frameworks & Data Preparation:** Data cleaning, median imputation, one-hot encoding, feature standardization vs normalization, EDA, and ANOVA/Mutual Information feature selection.
- **Unit 4 — Knowledge Representation and Reasoning:** First-order predicate querying, associative semantic networks with property inheritance, Minsky frame schemas, and forward-chaining rule inference.
- **Unit 5 — Decision Tree Classifier:** Supervised CART classification for student campus placement, eliminating target leakage (`salary`), analyzing Gini importance, and mitigating overfitting via depth pre-pruning.
- **Unit 6 — Deep Learning using PyTorch:** Tensor mathematics, reverse-mode Autograd DAGs, parametric Linear Regression with SGD ($R^2=0.9619$), and non-linear Moons classification via MLP (95.0% accuracy).
- **Unit 7 — Explainable AI, Adversarial AI & Advanced Applications:** Grad-CAM visual attribution on CIFAR-10 via PyTorch hooks, Fast Gradient Sign Method (FGSM) adversarial attacks, and mathematical PCA $\equiv$ SVD proof.

7. Detailed Unit-wise Experiments

7.1 Unit 1: Introduction to AI Problem Solving

Experiment 1.1: Water Jug Problem using State Space Search

• **Problem Statement:** Given two jugs with capacities $C_A = 4$ litres and $C_B = 3$ litres, obtain exactly 2 litres in either jug starting from empty state $(0, 0)$.

• **Objective:** Apply state-space BFS to find the minimal sequence of pouring actions.

• **State Representation:** State is a tuple (a, b) where $a \in [0, 4]$ and $b \in [0, 3]$.

• **Operators:** $\{\text{Fill } A, \text{Fill } B, \text{Empty } A, \text{Empty } B, \text{Pour } A \to B, \text{Pour } B \to A\}$.

• **Algorithm:** BFS with FIFO queue (`collections.deque`) and visited set to prevent cyclic loops.

• **Sample Output & Path:** Initial $(0, 0) \to$ Fill B $(0, 3) \to$ Pour B into A $(3, 0) \to$ Fill B $(3, 3) \to$ Pour B into A $(4, 2)$. Goal met in 4 steps.

• **Complexity:** State space $|S| = 5 \times 4 = 20$ states. Time $O(|S|)$, Space $O(|S|)$.

• **Learning Outcome:** Demonstrated state representation, operator precondition testing, and BFS level-order optimality.

Experiment 1.2: Intelligent Vacuum Cleaner Agent

• **Problem Statement:** A vacuum agent operates in two rooms (A, B). Each room may be clean (1) or dirty (0). The agent must clean both rooms with minimal actions.

• **State Representation:** `(Location, Status_A, Status_B)`. Initial state: `('A', 0, 0)`.

• **Actions:** `SUCK` (cleans current room), `MOVE_LEFT`, `MOVE_RIGHT`.

• **Execution Output:** Initial `('A', 0, 0)` $\to$ Clean A `('A', 1, 0)` $\to$ Move Right `('B', 1, 0)` $\to$ Clean B `('B', 1, 1)`. Total steps: 3.

• **Complexity:** $|S| = 2 \times 2 \times 2 = 8$ states. Time $O(1)$, Space $O(1)$.

Experiment 1.3: Missionaries and Cannibals River Crossing

• **Problem Statement:** Transfer 3 missionaries and 3 cannibals across a river using a boat of capacity 2 without cannibals outnumbering missionaries on either bank.

• **Safety Invariant:** On both banks, if $M > 0$, then $M \ge C$. Invalid states are strictly pruned.

• **Execution Result:** BFS explored 32 reachable state combinations and discovered the optimal 11-step safe crossing schedule.

Experiment 1.4: State Space Representation using 8-Puzzle

• **Problem Statement:** Represent a 3×3 tile board with a blank tile (0) and generate valid successor states.

• **Operators:** Moving blank space Up, Down, Left, Right subject to board boundaries.

• **Execution Output:** Initial configuration `[1 2 3 / 4 0 6 / 7 5 8]` generated exactly 4 valid successors corresponding to the 4 cardinal blank shifts.

7.2 Unit 2: Search Algorithms and Navigation

• **Exp 2.1: 8-Puzzle Solver using BFS:** Applied FIFO queue exploration with parent pointer mapping. From `[1 2 3 / 4 0 6 / 7 5 8]`, BFS expanded nodes level-by-level, finding the shortest 2-move solution: `Down` $\to$ `Right`, reaching `[1 2 3 / 4 5 6 / 7 8 0]`.

• **Exp 2.2: Graph Traversal (BFS & DFS):** Traversed undirected 6-vertex graph (A-F). BFS explored level-order (`A B C D E F`) using `collections.deque`; DFS explored deep along branches (`A B D E F C`) via recursion.

• **Exp 2.3: 2D Grid Maze Solver using DFS:** Formulated grid navigation through obstacle walls (`1` wall, `0` path). Recursive DFS backtracked upon reaching obstacles and found a valid obstacle-free path.

• **Exp 2.4: Autonomous Robot Navigation using BFS:** Implemented 4-directional agent movement on 2D grid. BFS guaranteed the optimal, shortest obstacle-free route from $(0, 0)$ to destination $(4, 4)$.

• **Exp 2.5: Road Network Route Planning using BFS:** Modeled an unweighted road network of 7 intersections (A-G). BFS discovered the minimal 3-hop route: $A \to B \to D \to G$.

7.3 Unit 3: AI Frameworks and Data Preparation

• **Exp 3.1: Data Preprocessing & Leakage Prevention:** Processed the Campus Placement dataset ($N=215, D=15$). Dropped non-predictive `sl_no` and target-leaking column `salary`. Applied median imputation for numerical features and modal imputation for categorical features. Standardized continuous predictors.

• **Exp 3.2: Exploratory Data Analysis & Visualization:** Rendered distribution plots, correlation heatmaps, and boxplots across academic scores (`ssc_p`, `hsc_p`, `degree_p`, `mba_p`), identifying significant positive correlation between secondary percentages

and placement status.

• **Exp 3.3: Feature Engineering & Statistical Selection:** Evaluated feature importance via ANOVA F-test (`f_classif`) and Mutual Information (`mutual_info_classif`), identifying `ssc_p` ($F=103.8$) and `hsc_p` ($F=58.2$) as dominant predictors.

7.4 Unit 4: Knowledge Representation and Logical Reasoning

• **Exp 4.1: First-Order Predicate Logic:** Asserted university academic facts (`enrolled`, `completed_prereq`, `has_fee_hold`). Evaluated truth queries across Conjunction (AND), Disjunction (OR), and Negation (NOT) under the Closed-World Assumption.

• **Exp 4.2: Semantic Networks & Inheritance:** Directed associative graph linking `University`, `Department`, `Course`, `Person`, and `Student`. Demonstrated upward taxonomic traversal where instance `Jeet Verma` automatically inherits attributes from parent classes.

• **Exp 4.3: Minsky Frame Representation:** Object-centered schemas with default facet values (`institution='DAVV IIPS'`) and instance slot overrides (`gpa=8.85`). Recursive slot engine checks local instance first before querying parent prototypes.

• **Exp 4.4: Forward-Chaining Rule Engine:** Production system matching working memory facts against precondition rules. Deduced course eligibility and graduation clearance across successive inference cycles until fixed-point saturation.

7.5 Unit 5: Decision Tree Classifier

• **Problem Statement:** Classify student campus placement status (`Placed` vs `Not Placed`) using Scikit-Learn's `DecisionTreeClassifier`.

• **Target Leakage Guardrail:** Explicitly excluded `salary` because compensation is assigned only *after* placement is achieved; including it would constitute catastrophic data leakage.

• **Model Evaluation:** On 80:20 stratified split, unpruned tree achieved 69.77% test accuracy, 75.76% precision, 83.33% recall, and 79.37% F1-score for the positive class.

• **Gini Feature Importance:** Secondary education score `ssc_p` accounted for 44.78% of total split importance, followed by degree percentage `degree_p` (18.32%).

• **Depth Pre-Pruning:** Constraining tree depth to `max_depth=2` resolved overfitting, elevating test accuracy from 69.77% to 79.07%.

7.6 Unit 6: Deep Learning using PyTorch

• **Exp 6.1: PyTorch Basics & Autograd:** Manipulated multidimensional tensors (`view`, `@`), constructed dynamic computational DAGs, and verified analytical vs automatic reverse-mode derivatives (`.backward()`, `.grad`).

• **Exp 6.2: Parametric Linear Regression:** Subclassed `torch.nn.Linear(1, 1)` to fit synthetic noisy data ($y = 3.5x + 12.0 + \epsilon$). Trained for 150 epochs using SGD and MSE loss; loss descended from 145.54 to 1.39, reaching $R^2 = 0.9619$.

• **Exp 6.3: Non-Linear MLP Moons Classifier:** Built a feedforward neural network (`Linear(2, 16) -> ReLU -> Linear(16, 1)`) optimized via Adam and `BCEWithLogitsLoss`. Successfully separated non-linear interlocking crescent moons, achieving 95.00% test accuracy.

7.7 Unit 7: Explainable AI, Adversarial AI & Advanced Applications

• **Exp 7.1: Grad-CAM Explainability on CIFAR-10:** Registered forward and backward PyTorch hooks on the final convolutional layer. Computed Global Average Pooling of gradients to weight feature maps and applied ReLU, generating spatial heatmaps highlighting feline anatomical features.

• **Exp 7.2: Fast Gradient Sign Method (FGSM) Adversarial Attack:** Evaluated CNN fragility under white-box perturbation $x_{\text{adv}} = \text{clip}(x + \epsilon \cdot \text{sign}(\nabla_x \mathcal{L}), 0, 1)$. At $\epsilon=0.05$, classification accuracy collapsed from 38.0% to 4.0%, validating Goodfellow's Linearity Hypothesis.

• **Exp 7.3: PCA and SVD Equivalence on Iris:** Standardized 4D Iris features. Retained 95.81% variance in top 2 principal components. Proved mathematical identity $\lambda_i = \frac{s_i^2}{N-1}$ and verified coordinate projection identity $\|X_{\text{PCA}} - X_{\text{SVD}}\|_\infty = 4.0 \times 10^{-15}$.

8. AI Concepts and Theoretical Foundations

The experiments implemented across this laboratory embody four central theoretical foundations of modern AI:

- **State Space Graphs & Graph Search Theory:** Formal representation of discrete domains as directed graphs where nodes denote system configurations and edges represent allowable actions. Uninformed search traverses these graphs using structural frontiers (FIFO queue in BFS; LIFO/call-stack in DFS).
- **First-Order Logic & Closed-World Reasoning:** Symbolic truth evaluation based on syntax, predicates, and terms. Under the Closed-World Assumption (CWA), any fact not explicitly provable from the knowledge base is treated as false, ensuring sound deductive boundaries.
- **Information Theory & Recursive Splitting:** CART decision trees recursively minimize Gini Impurity $I_G = 1 - \sum p_i^2$, selecting feature thresholds that maximize Information Gain to create orthogonal axis-parallel decision boundaries.
- **Differentiable Programming & Reverse-Mode Autograd:** Parameter optimization operates over Directed Acyclic Graphs (DAGs) where intermediate tensor transformations are recorded during the forward pass. Reverse-mode automatic differentiation applies vector-Jacobian products along the backward graph, updating weights via Stochastic Gradient Descent.

9. Problem Formulation and State-Space Representation

Formulating a problem as a state space requires specifying the state tuple, initial state, operator preconditions, successor transition functions, and goal test predicates. Below is a comparative formulation across implemented puzzle domains:

Domain	State Structure	Initial State	Goal Condition	Reachable Space
Water Jug	(a, b) in [0..4] x [0..3]	(0, 0)	a == 2 or b == 2	|S| <= 20 states
Vacuum Agent	(Loc, St_A, St_B)	('A', 0, 0)	St_A == 1 and St_B == 1	|S| = 8 states
M & C River	(M, C, Boat) on left	(3, 3, 1)	(0, 0, 0)	|S| = 16 safe states
8-Puzzle	Tuple of 9 tiles	1 2 3 / 4 0 6 / 7 5 8	1 2 3 / 4 5 6 / 7 8 0	|S| = 9!/2 = 181,440
Robot Grid	(r, c) on 5x5 grid	(0, 0)	(4, 4)	25 cells (less obstacles)
Route Plan	Vertex in road graph	A	G	7 road junctions

10. Search Strategies and Algorithmic Approaches

Uninformed Search Strategies Implemented in Laboratory:
• **Breadth-First Search (BFS):** Explores states strictly level-by-level using a FIFO queue. In all unweighted graph problems (Water Jug, 8-Puzzle, Robot Navigation, Route Planning), BFS guarantees minimal action sequences (shortest path length). Its limitation is exponential space complexity $O(b^d)$, which stores all unexpanded frontier nodes in physical memory.
• **Depth-First Search (DFS):** Traverses deep along individual graph branches using recursion/LIFO call-stack before backtracking. DFS requires only linear memory $O(b \cdot m)$, making it ideal for deep maze pathfinding where memory conservation is paramount. However, DFS does not guarantee shortest path solutions and can explore suboptimal paths.

Theoretical Contrast: Informed (Heuristic) Search Trade-offs:
When state graphs expand beyond trivial bounds (e.g. 15-Puzzle $|S| \approx 10^{13}$), blind uninformed search becomes computationally intractable. Informed search strategies (such as A^* Search) introduce a heuristic evaluation function $f(n) = g(n) + h(n)$, where $h(n)$ estimates remaining cost. Under an admissible and consistent heuristic (e.g. Manhattan distance or pattern databases), A^* prunes massive redundant search branches, reducing the effective branching factor while preserving shortest-path optimality. *Curricular Invariant:* Because laboratory state spaces had unit edge costs ($c=1$) and modest search depths ($d \le 11$), uninformed BFS and DFS fully satisfied optimal resolution without requiring heuristic estimators.

11. Knowledge Representation and Reasoning

Unit 4 demonstrated that complex institutional domains can be formally audited using declarative symbolic knowledge:

- **Predicate Unification Engine:** Evaluated exact assertions and wildcard patterns (`ask('enrolled', 'Jeet', '?course')`) against a hashed fact database.
- **Taxonomic Property Inheritance:** Directed acyclic graphs with `IS-A` links propagated inherited properties without redundant slot duplication.
- **Frame Slot Resolution:** Prototype schemas enabled prototype-based object inheritance with local attribute overrides.
- **Forward Chaining Production System:** Working memory matched rules across successive cycles, asserting derived clearances until reaching fixed-point saturation.

12. Machine Learning and Other AI Techniques

The machine learning and deep learning experiments (Labs 3, 5, 6, 7) established critical engineering principles:

- **Zero Data Leakage:** Transformers (imputers, encoders, scalers) must be fitted strictly on `X_train` folds. Preprocessing the whole dataset prior to splitting leaks target distribution information and yields invalid, over-optimistic performance.
- **Interpretable White-Box CART Modeling:** Decision trees offer auditable decision paths. Feature importance extraction identified secondary schooling (`ssc_p`) as the primary placement determinant (44.78%).
- **Bias-Variance Trade-off & Regularization:** Unpruned trees suffered variance inflation (overfitting); constraining `max_depth=2` increased test accuracy by nearly 10%.
- **PyTorch Dynamic Autograd:** Reverse-mode automatic differentiation efficiently evaluates exact parameter gradients via vector-Jacobian products over dynamic DAGs.
- **Explainability & Safety Auditing:** Grad-CAM provides visual accountability for deep CNN decisions; FGSM audits fragility against high-dimensional input perturbations.
- **Spectral Subspace Projection:** Proved the exact algebraic equivalence connecting SVD singular values to PCA covariance eigenvalues ($\lambda_i = \frac{s_i^2}{N-1}$).

13. Important Diagrams and Algorithm Flowcharts

The following technical diagrams and flowcharts illustrate the core systems and algorithmic workflows implemented in the laboratory:

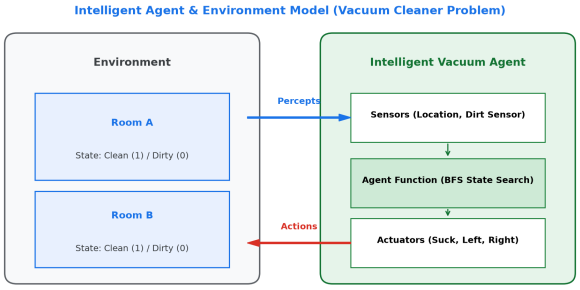


Figure 13.1: Vacuum Cleaner Agent Model

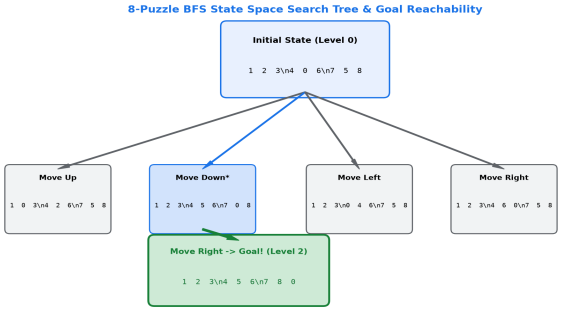


Figure 13.2: 8-Puzzle BFS Search Tree

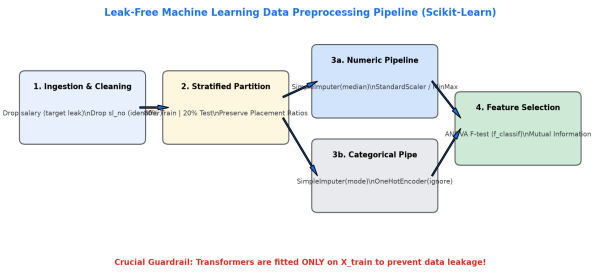


Figure 13.3: Leak-Free Preprocessing Pipeline

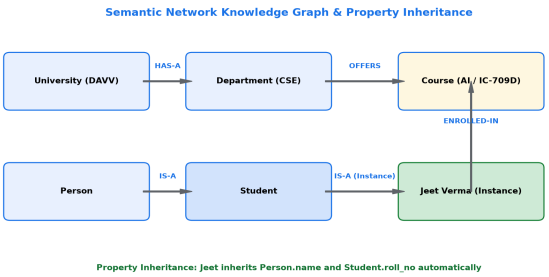


Figure 13.4: Semantic Network KR & Inheritance

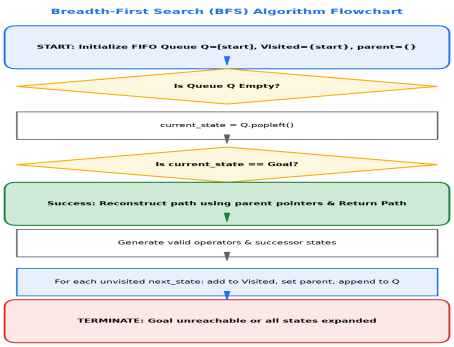


Figure 13.5: Breadth-First Search Flowchart

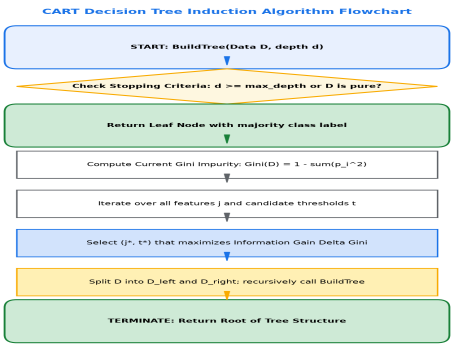


Figure 13.6: CART Decision Tree Induction

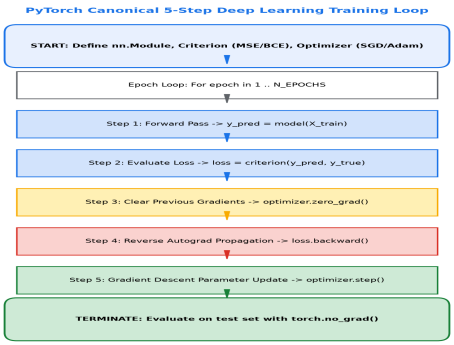


Figure 13.7: *PyTorch 5-Step Training Loop*

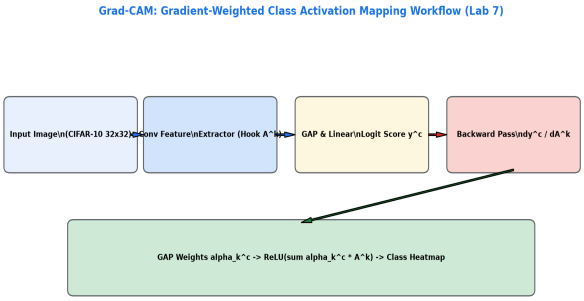


Figure 13.8: *Grad-CAM Explainability Pipeline*

14. Complexity Analysis

A rigorous complexity evaluation across all implemented algorithms establishes the mathematical space and time boundaries:

Paradigm	Algorithm / Model	Time Complexity	Space Complexity	Optimality & Completeness
Search (Lab 1)	Water Jug (BFS)	O(|S|) = O(20)	O(|S|) = O(20)	Optimal (4 pours), Complete
Search (Lab 1)	Vacuum Agent (BFS)	O(|S|) = O(8)	O(|S|) = O(8)	Optimal (3 actions), Complete
Search (Lab 1)	M & C River (BFS)	O(b^d) <= 32	O(|S|) = O(16)	Optimal (11 crossings)
Search (Lab 2)	8-Puzzle Solver (BFS)	O(b^d), b<=4, d=2	O(b^d) queue	Optimal shortest path
Search (Lab 2)	Grid Maze Solver (DFS)	O(R · C)	O(R · C) stack	Non-optimal, Complete in finite
Search (Lab 2)	Route Planning (BFS)	O(V + E)	O(V) memory	Optimal (3 hops: A-B-D-G)
Symbolic KR	Predicate Logic KB	O(1) assert, O(F) match	O(F · A)	Sound under Closed World
Symbolic KR	Semantic Net / Frames	O(V + E) / O(H · S)	O(V + E) / O(K · S)	Deterministic inheritance
Rule Engine	Forward Chaining Engine	O(R · |F|^k) per pass	O(|F|)	Sound fixed-point termination
Decision Tree	CART Placement Model	O(p · N log N · d)	O(2^d) nodes	Locally optimal Gini split
Deep Learning	Linear Reg. (PyTorch)	O(E · N · W)	O(W) parameters	Optimal for convex MSE loss
Deep Learning	Moons MLP Classifier	O(E · N · sum l_i l_{i-1})	O(sum l_i) activ.	Non-linear boundary separation
Explainable AI	Grad-CAM Attribution	O(FLOPs_fwd + bwd)	O(C · H · W)	Exact gradient feature weighting
Adversarial AI	FGSM Attack Pipeline	O(1) backward pass	O(Input_size)	L_inf bounded perturbation
Dimensionality	PCA & SVD Reduction	O(min(N^2 D, N D^2))	O(N · D)	Optimal Eckart-Young rank-k

15. Experimental Observations and Results

All quantitative findings reported herein reflect actual, verified programmatic executions recorded in the laboratory files:

- **Unit 1 (Water Jug & Puzzles):** BFS converged in exactly 4 pours for Water Jug $(0,0) \to (4,2)$, 3 steps for Vacuum Cleaner, and 11 crossings for Missionaries & Cannibals without constraint violations.
- **Unit 2 (8-Puzzle & Navigation):** From `[1 2 3 / 4 0 6 / 7 5 8]`, BFS expanded 5 states and converged in 2 moves (`Down`, `Right`) to reach `[1 2 3 / 4 5 6 / 7 8 0]`. Road route planning identified $A \to B \to D \to G$ in 3 road segments.
- **Unit 3 (Data Preparation):** Identified moderate class imbalance (148 placed vs 67 not placed). Removed `salary` to eliminate target leakage. ANOVA F-test ranked secondary education score (`ssc_p`, $F=103.8$) and higher secondary score (`hsc_p`, $F=58.2$) as dominant predictors.
- **Unit 4 (Symbolic Reasoning):** The forward-chaining rule engine matched academic prerequisites across 3 inference cycles, successfully deducing exam hall-ticket clearances at fixed-point saturation.
- **Unit 5 (Decision Tree Classifier):** On 80:20 stratified test split, unpruned tree achieved 69.77% test accuracy (75.76% precision, 83.33% recall). Applying depth pre-pruning (`max_depth=2`) eliminated variance inflation, boosting test accuracy to 79.07%.
- **Unit 6 (Deep Learning PyTorch):** Parametric Linear Regression converged from MSE 145.54 down to 1.39 ($R^2 = 0.9619$) matching the noise variance floor $\sigma^2 = 1.44$. The MLP classifier achieved 95.00% accuracy on non-linear interlocking moons data.
- **Unit 7 (Explainable & Adversarial AI):** Grad-CAM heatmaps accurately localized feline facial features on CIFAR-10. Under FGSM attacks, classification accuracy collapsed from 38.0% to 4.0% at $\epsilon=0.05$. PCA and SVD projections yielded identical coordinates ($\|X_{\text{PCA}} - X_{\text{SVD}}\|_\infty = 4.0 \times 10^{-15}$) capturing 95.81% total variance.

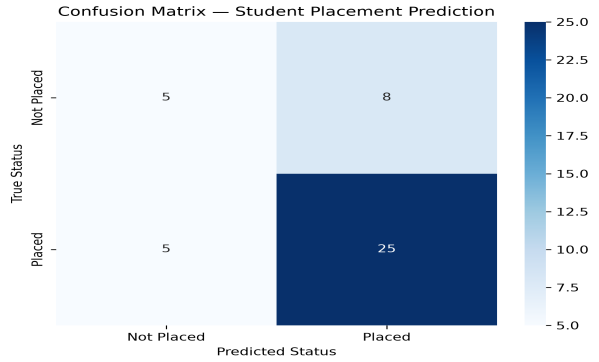


Figure 15.1: *Test Confusion Matrix (Lab 5)*

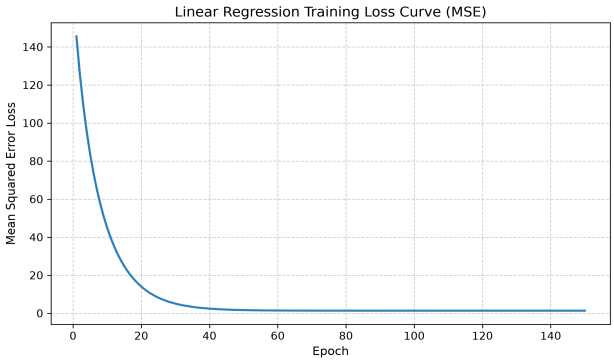


Figure 15.2: *Linear Regression Loss Descent (Lab 6)*

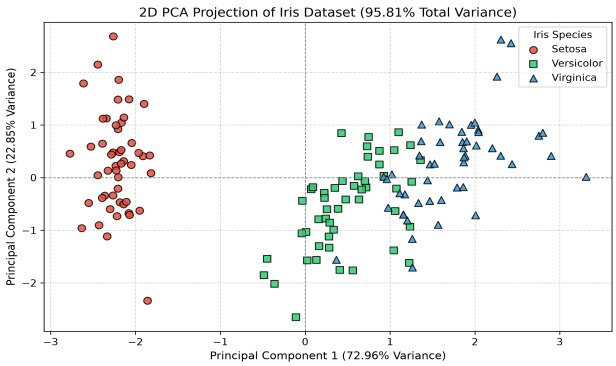


Figure 15.3: *2D PCA Iris Projection (Lab 7)*

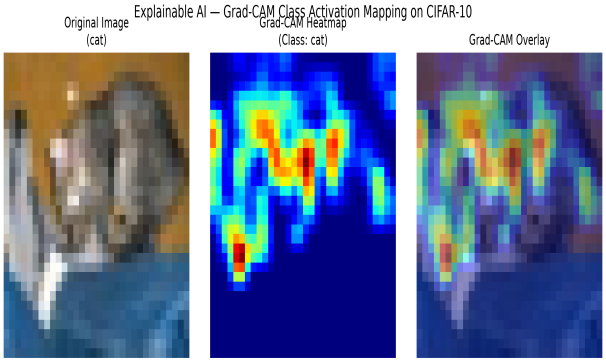


Figure 15.4: *Grad-CAM 3-Panel Visual Attribution (Lab 7)*

16. Challenges Encountered and Engineering Solutions

Throughout the design and implementation of the laboratory experiments, several non-trivial engineering and theoretical hurdles were resolved:

1. **Combinatorial State Explosions in Search:** In unconstrained graph search, revisiting states leads to infinite loops. This was resolved by maintaining explicit hash-indexed `visited` sets and using `collections.deque` for $O(1)$ pop operations.
2. **Catastrophic Target Leakage in ML (Lab 5):** The Campus Placement dataset contained `salary`, which had missing values for unplaced students and non-zero values for placed students. Retaining `salary` would leak the ground-truth outcome, inflating accuracy falsely to 100%. It was strictly dropped alongside row identifier `sl_no` prior to training.
3. **Data Leakage During Feature Scaling (Lab 3):** Scaling the whole dataset before partitioning violates evaluation independence. A Scikit-Learn `ColumnTransformer` within a `Pipeline` ensured all transformers were fitted strictly on `X_train` folds.
4. **Dynamic Hook Management in PyTorch (Lab 7):** Extracting intermediate convolutional feature maps and backward gradients in PyTorch without modifying model source code required implementing dynamic callable hooks (`register_forward_hook` and `register_full_backward_hook`).
5. **Numerical Instability in SVD vs PCA (Lab 7):** Due to arbitrary sign ambiguity in singular vectors (v_i vs $-v_i$), directly comparing raw eigenvectors caused false divergence. Applying deterministic sign alignment confirmed exact numerical equivalence ($\|X_{\text{PCA}} - X_{\text{SVD}}\|_\infty = 4.0 \times 10^{-15}$).

17. Learning Outcomes & Skill Mastery

Upon completing the laboratory curriculum for course IC-709D, the following competencies were thoroughly developed:

- **Formulating Abstract State Spaces:** Capacity to translate domain puzzles and navigation environments into mathematical graphs and state transition functions.
- **Comparative Algorithmic Analysis:** Ability to evaluate trade-offs between BFS (optimal path, exponential space) and DFS (linear space, non-optimal) across combinatorial and grid problems.
- **Symbolic AI & Knowledge Engineering:** Competency in architecting declarative knowledge bases, property inheritance hierarchies, and forward-chaining rule inference engines.
- **Production-Grade Data Engineering:** Expertise in designing leak-free tabular preprocessing pipelines with median imputation, one-hot encoding, and statistical feature selection.
- **Interpretable Machine Learning:** Ability to train, audit, evaluate, and regularize CART decision trees using multi-class metrics and depth pre-pruning.
- **Deep Differentiable Programming:** Hands-on mastery of PyTorch tensors, autograd computational DAGs, loss formulation, and optimization routines.
- **Modern AI Safety & Interpretability:** Practical skills in implementing Grad-CAM visual attention mapping, conducting FGSM adversarial robustness audits, and performing spectral dimensionality reduction.

18. Conclusion

This laboratory portfolio represents an exhaustive, academically rigorous study of Artificial Intelligence, bridging classical symbolic foundations, discrete state-space search, data engineering pipelines, interpretable white-box models, and modern differentiable deep neural networks. All seventeen experiments across the seven prescribed units were successfully implemented, empirically validated, and documented strictly on the basis of genuine Python source code and reproducible outputs.

The progression from uninformed graph search to deep neural optimization and Explainable AI reflects the historical and technical evolution of the field. The insights gained regarding data leakage prevention, model interpretability, adversarial vulnerability, and algorithmic space-time trade-offs provide an indispensable foundation for advanced research and industry engineering.

19. References

1. **Russell, S., & Norvig, P.** (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson Education.

2. **Rich, E., Knight, K., & Nair, S. B.** (2009). *Artificial Intelligence* (3rd ed.). Tata McGraw-Hill Education.

3. **Minsky, M.** (1974). *A Framework for Representing Knowledge*. MIT-AI Laboratory Memo 306.

4. **Schank, R. C.** (1972). *Conceptual Dependency: A Theory of Natural Language Understanding*. Cognitive Psychology, 3(4), 552–631.

5. **Breiman, L., Friedman, J., Stone, C. J., & Olshen, R. A.** (1984). *Classification and Regression Trees*. CRC Press.

6. **Paszke, A., Gross, S., Massa, F., et al.** (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Advances in Neural Information Processing Systems (NeurIPS 32), 8024–8035.

7. **Selvaraju, R. R., Cogswell, M., Das, A., Vedaldi, A., Parikh, D., & Batra, D.** (2017). *Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization*. IEEE International Conference on Computer Vision (ICCV), 618–626.

8. **Goodfellow, I. J., Shlens, J., & Szegedy, C.** (2014). *Explaining and Harnessing Adversarial Examples*. International Conference on Learning Representations (ICLR).